Tower & Slack Pack Integration Runbook

Runbook — Tower & Slack Pack Integration (TASK-013 → TASK-018)

Scope: manual + scripted verification of the six modules merged in the TOWER pack-integration wave: `commands.py`, `tower_sync.py`, `slack_notify.py`, `slack_listener.py`, `inbox.py`, and their wiring into `supervisor.py`.

Status of the feature at rest: everything in this wave ships **disabled by default** (`tower.enabled: false`, `slack.enabled: false` in `autopilot.json`). A healthy tick with both disabled is byte-identical to pre-wave behaviour — so Suite A (regression) and Suite G.1 (disabled-parity) are safe to run against any environment, including production, with zero risk. Suites B–F and G.2 require deliberate opt-in and should be run against a throwaway project or a scratch branch, never a live production supervisor, until you've completed this runbook once.

Audience: whoever is validating this wave before flipping `enabled: true` on a real project (Alister, or a delegated reviewer).

---

0. Prerequisites

Requirement	Notes
Python 3.10+, this repo checked out, `pip install -e .`-equivalent deps already present	No new *required* dependency — `slack_sdk` is optional (see 0.1)
`pytest`, Node (for `hooks/run-tests.js`)	Same as any other regression run
A Slack workspace where you can create an app (Suites C/D only)	Not needed for Suites A/B/E/F/G.1
A reachable Tower HTTP endpoint, or a local stub (Suite E only)	See 5.1 for a 10-line stub server
A throwaway/test PLAN.md and worktree, **not** a live production supervisor	Suites F and G.2 write files and start background threads

0.1 — Optional dependency: `slack_sdk`

Only `slack_listener.py` needs it, and only to actually connect. Everything else in this wave has zero third-party runtime dependencies (stdlib `urllib` only).

```
python -c "import slack_sdk" 2>&1   # if this errors, slack_sdk is absent — expected on a fresh
        checkout
pip install slack_sdk               # only needed for Suite D's live-connection step
```

Confirm the **absent-dependency fail-open path** first, before installing anything — this is the default state most projects will run in:

```
python -m pytest tests/test_slack_listener.py -k "dependency or unavailable or not_installed" -v
```

Expect all such tests to **pass** against the real absent environment. If `slack_sdk` is already installed on your machine, this specific check is skipped rather than failed — that's fine, it just means you'll validate the absent-path in CI/on a clean box instead.

0.2 — Env vars reference (never in a tracked file)

Var	Used by	Format	Scope
`DEVTEAM_TOWER_TOKEN`	`tower_sync.py`	bearer token, any string	per-project
`DEVTEAM_SLACK_TOKEN`	`slack_notify.py`, `slack_listener.py`	`xoxb-...` (Bot User OAuth Token)	per-workspace
`DEVTEAM_SLACK_APP_TOKEN`	`slack_listener.py` (Socket Mode)	`xapp-...`	per-workspace (shared across projects on the same workspace)
`DEVTEAM_SLACK_SIGNING_SECRET`	Tower-side request verification (not this repo)	hex	per-workspace

Set these in your shell / PM2 env config for the test session. **Never** commit them, and never let a builder or this runbook write them to a tracked file.

Suite A — Automated regression (run first, every time)

This is the fastest signal and should be green before you touch anything manual.

```
python -m pytest -q
node hooks/run-tests.js
```

Expected: full green — at the time this wave closed, **931 passed / 0 failed** (pytest) and **36 passed / 0 failed** (node). Your exact count will grow as the pack evolves; what matters is **0 failed**.

If anything fails here, stop — do not proceed to the manual suites below until the regression suite is clean.

Suite B — `commands.py` vocabulary validation

Confirms the shared command validator (used by both Tower's inbox and Slack's listener) rejects what it should and accepts what it should, with no second/duplicate validator anywhere else in the pack.

```
python - <<'PY'
from scripts import commands

# Known-good vocabulary
for cmd, args in [
    ("approve", {"task_id": "TASK-001"}),
    ("rework", {"task_id": "TASK-001", "text": "fix X"}),
    ("stop", {}),
    ("resume", {}),
    ("wave", {}),
]:
    ok, result = commands.validate({"command": cmd, "args": args})
    print(cmd, "->", ok, result)

# Known-bad: unknown command
print(commands.validate({"command": "frobnicate", "args": {}}))

# Known-bad: typo must NOT be silently corrected to the nearest command
print(commands.validate({"command": "aprove", "args": {"task_id": "TASK-001"}}))
PY
```

Expected: - Every listed known-good command validates `True` with a clean `{command, args}` pair back. - `frobnicate` → `(False, "unknown command")` (exact wording may differ — the point is `False`, never guessed). - `aprove` (typo) → **also rejected**, not rewritten to `approve`. If a typo is ever silently accepted as its "nearest" command, that's a regression — file it immediately, this was a named test case (`test_typo_is_not_rewritten_to_nearest_command`) precisely to prevent it.

Suite C — `slack_notify.py` (outbound messages)

C.1 — Config-only dry check (no Slack app needed yet)

```
python scripts/slack_notify.py --test
```

With `slack.enabled: false` / no channels configured (the shipped default), expect:

```
[slack_notify] --test: no channels configured (autopilot.json slack.ops_channel /
slack.project_channel)
```

This confirms the fail-open no-op path — the smoke test does not error, it tells you plainly why it did nothing.

C.2 — Live smoke test (Slack §10's required checklist)

Before enabling on any real project, a human must complete all five steps — do not skip any of them:

1. **Create the Slack app** from the manifest in `specs/DEVDEPARTMENT_SLACK_SPEC.md`.
2. **Invite the bot** to both the ops channel and the project channel.
3. **Set all three env vars** in the shell / PM2 config that will run the supervisor: `DEVTEAM_SLACK_TOKEN`, `DEVTEAM_SLACK_APP_TOKEN`, `DEVTEAM_SLACK_SIGNING_SECRET`.
4. Fill in `autopilot.json`'s `slack.ops_channel` / `slack.project_channel` (channel IDs, not names) on your **test** project, then run:

   ```
   python scripts/slack_notify.py --test --repo /path/to/test/project
   ```

5. Confirm delivery **by eye** — one message should land in each configured channel: ✅ `DEVDEPARTMENT slack_notify.py --test smoke message (<label>)`.

Only after all five steps pass should `slack.enabled: true` be set and `"slack"` added to `notify_channels` on a real project.

C.3 — Block Kit spot-check

Trigger each of the four real notification shapes at least once and visually confirm the buttons/layout match the spec (§3): `P2-NEEDS-REVIEW`, `P2-BLOCKED`, `P1-STOP-THE-LINE`, `P0-WAVE-COMPLETE`. The easiest way is to drive them through the normal supervisor flow on a disposable test project (flip a task to `needs_review`, trigger a stop-the-line condition, etc.) rather than calling internals directly — you're checking what a real operator sees.

Also confirm **thread behaviour**: a decision on a P2-NEEDS-REVIEW message should update the *same* Slack thread (via `chat.update`), not post a new message, and a completed task should get a ✅ reaction on its original post.

Suite D — `slack_listener.py` (inbound Socket Mode commands)

D.1 — Absent-dependency path (default state)

With `slack_sdk` **not installed** (the default), confirm the listener refuses cleanly:

```
python - <<'PY'
from scripts.slack_listener import SlackListener
import queue
l = SlackListener(app_token="xapp-fake", bot_token="xoxb-fake", out_queue=queue.Queue())
l.start()
print("available:", l.available)
print("is_alive:", l.is_alive())
PY
```

Expected: exactly one warning line naming `pip install slack_sdk`, `available` is `False`, `is_alive()` is `False` — no thread was spawned, no exception raised, and every other channel (Telegram, console, Slack outbound) is completely unaffected.

D.2 — Live connection (requires `slack_sdk` installed + real tokens)

```
pip install slack_sdk
```

Set `DEVTEAM_SLACK_APP_TOKEN` / `DEVTEAM_SLACK_TOKEN`, add `"slack"` to a **test** project's `notify_channels`, then start the supervisor once:

```
python scripts/supervisor.py --once --repo /path/to/test/project
```

Expect `[supervisor] Slack listener started.` on stdout/stderr. From Slack, issue a slash command (e.g. `/devteam status`) in the channel the app is installed to. Confirm: - The command is **acked immediately** (Slack shows no retry/error). - On the next supervisor tick, the command shows up drained and routed through the same handler Telegram commands use (check `AUTOPILOT_LOG.md` or the console for the resulting action). - An **unknown** command name (e.g. `/devteam frobnicate`) is still enqueued by the listener (it does not judge vocabulary) but is then rejected by the shared drain/`commands.py` on the next tick — confirms Suite B's contract holds end-to-end, not just in isolation. - A non-slash-command Slack event (e.g. clicking a button) is acked but not enqueued — that's Tower's territory (§6), not this listener's.

Suite E — `tower_sync.py` (snapshot push + queue pull)

E.1 — Local stub server (no real Tower needed)

Save as `tower_stub.py` and run in a separate terminal:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", 0))
        body = json.loads(self.rfile.read(length) or b"{}")
        print("INGEST received, schema:", body.get("schema"), "tasks:", len(body.get("tasks",
         [])))
        self.send_response(200); self.end_headers()

    def do_GET(self):
        print("QUEUE pull for", self.path)
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps([
            {"id": "cmd-1", "issued_at": "2026-08-26T12:00:00Z", "source": "tower",
             "actor": "alister", "command": "status", "args": {}}
        ]).encode())

    def do_DELETE(self):
        print("ACK delete for", self.path)
        self.send_response(200); self.end_headers()

HTTPServer(("127.0.0.1", 8765), Handler).serve_forever()
```

```
python tower_stub.py &
```

On your **test** project, set `tower.enabled: true`, `tower.url: "http://127.0.0.1:8765"`, `tower.project_id: "test-project"`, and `DEVTEAM_TOWER_TOKEN=anything`:

```
python scripts/supervisor.py --once --repo /path/to/test/project
```

Expected: - Stub prints `INGEST received, schema: 1, tasks: N` — confirms `build_snapshot()` produced a well-formed v1 snapshot. - Stub prints `QUEUE pull for /queue/test-project` and `ACK delete for /queue/test-project/cmd-1`. - One new file materializes at `<test-project>/.devteam/inbox/cmd-1.json` (or similarly named) with the command payload from the stub.

E.2 — Fail-open verification (the important one)

Kill the stub server (`kill %1` or Ctrl-C it), then run the tick again:

```
python scripts/supervisor.py --once --repo /path/to/test/project
```

Expected: exactly one `[tower] warning: ...` line (connection refused/timeout), the tick completes normally, no exception, no crash, and every other subsystem (Slack, console, Telegram) is unaffected. This is

the single most important thing to verify before ever enabling Tower against a real, possibly-flaky network endpoint — a Tower outage must never be able to take down a project's autopilot.

Also verify the disabled case is a silent no-op:

```
# tower.enabled: false, or tower.url: "" on the test project
python scripts/supervisor.py --once --repo /path/to/test/project
# expect: zero tower-related output at all, not even a warning
```

Suite F — `inbox.py` (drain, validate, reject, ack)

F.1 — Happy path

```
mkdir -p /path/to/test/project/.devteam/inbox
cat > /path/to/test/project/.devteam/inbox/good-1.json <<'EOF'
{"id": "good-1", "issued_at": "2026-08-26T12:00:00Z", "source": "tower",
 "actor": "alister", "command": "status", "args": {}}
EOF

python - <<'PY'
from pathlib import Path
from scripts import inbox
repo = Path("/path/to/test/project")
items = inbox.drain_inbox(repo, cfg={})
print("drained:", items)
print("still on disk after drain (should be True — draining != ack):",
      (repo / ".devteam/inbox/good-1.json").exists())
for item in items:
    inbox.ack(repo, item)
print("still on disk after ack (should be False):",
      (repo / ".devteam/inbox/good-1.json").exists())
PY
```

F.2 — Rejection path

```
echo 'not valid json{{{' > /path/to/test/project/.devteam/inbox/bad-1.json
cat > /path/to/test/project/.devteam/inbox/bad-2.json <<'EOF'
{"id": "bad-2", "command": "frobnicate", "args": {}}
EOF

python - <<'PY'
from pathlib import Path
from scripts import inbox
repo = Path("/path/to/test/project")
print(inbox.drain_inbox(repo, cfg={}))
PY

ls /path/to/test/project/.devteam/inbox/rejected/
```

Expected: both bad files are **moved** (never deleted) to `.devteam/inbox/rejected/`, each with a `.reason` sidecar explaining why. `good-1.json` is untouched by this run (already acked in F.1).

F.3 — Duplicate id rejection

Re-drop a file with an `id` that's already been acked (`good-1` from F.1) and confirm the second occurrence is rejected as a duplicate — check that this survives a fresh Python process (i.e., the consumed-id ledger is durable on disk, not an in-memory set that resets):

```
cat > /path/to/test/project/.devteam/inbox/good-1-again.json <<'EOF'
{"id": "good-1", "issued_at": "2026-08-26T12:05:00Z", "source": "tower",
 "actor": "alister", "command": "status", "args": {}}
EOF
python - <<'PY'
from pathlib import Path
from scripts import inbox
print(inbox.drain_inbox(Path("/path/to/test/project"), cfg={}))
PY
ls /path/to/test/project/.devteam/inbox/rejected/   # should now include good-1-again with a
        "duplicate" reason
```

F.4 — Crash-recovery (two-phase ack) simulation

Drain a valid command but **do not** call `ack()` — simulate a crash between drain and handling — then drain again and confirm the file is still returned (not lost):

```
cat > /path/to/test/project/.devteam/inbox/crash-1.json <<'EOF'
{"id": "crash-1", "issued_at": "2026-08-26T12:10:00Z", "source": "tower",
 "actor": "alister", "command": "status", "args": {}}
EOF
python - <<'PY'
from pathlib import Path
from scripts import inbox
repo = Path("/path/to/test/project")
first = inbox.drain_inbox(repo, cfg={})     # simulate crash here — never ack
second = inbox.drain_inbox(repo, cfg={})    # "restart"
print("survived the crash:", any(i["id"] == "crash-1" for i in second))
PY
```

Expected: `True` — a crash between drain and handling must never lose a command.

Suite G — `supervisor.py` end-to-end integration

G.1 — Disabled-parity (run this one against production-adjacent configs too — it's the safety net)

With `tower.enabled: false`, `slack` absent from `notify_channels`, and no `.devteam/inbox/` directory:

```
python scripts/supervisor.py --once --dry-run --repo /path/to/any/project
```

Expected: output is **behaviourally identical** to a pre-wave supervisor run — no `[tower]` lines, no `[supervisor] Slack listener` lines, no inbox activity of any kind. This is the graded acceptance criterion this wave was held to ("byte-identical when disabled") — if you see any trace of the new subsystems here, that's a regression.

G.2 — All three enabled together (the full integration path)

Combine E.1 (Tower stub running), D.2 (Slack listener + a test slash command), and F.1 (a command sitting in `.devteam/inbox/`) on the same test project, then:

```
python scripts/supervisor.py --once --repo /path/to/test/project
```

Expected, in order, within one tick: 1. Tower push+pull happens (stub logs `INGEST` / `QUEUE` / `ACK`). 2. Any command Tower's queue pull just materialized into `.devteam/inbox/`, **plus** any pre-existing inbox file, is drained via `inbox.drain_inbox()` **before** `decide()` runs. 3. Both the Telegram queue and the Slack queue (if a listener is running) are drained through the **same** handler path (`drain_command_queue`) — confirm via console/log output that a command issued from Slack and a command issued from Telegram produce identical downstream effects for the same command type. 4. Nothing about this crashes or hangs the tick even if one of the three sources (Tower/Slack/inbox) is simultaneously misbehaving — re-run E.2's fail-open

check with the Slack listener and inbox commands also active, to confirm one subsystem's failure doesn't take down the others.

Rollback / disable

Every piece of this wave is designed to be turned off instantly and safely:

```
// autopilot.json
"tower": { "enabled": false, ... },
"slack": { "enabled": false, ... }
```

and remove `"slack"` from `notify_channels`. No migration, no data loss — `.devteam/inbox/` and its `rejected/` subfolder can be left in place or cleared; nothing else reads them when disabled.

Troubleshooting

Symptom	Likely cause	Check
`slack_notify.py --test` says "no channels configured"	Expected default — Suite C.2 step 4 not done yet	Fill in `slack.ops_channel` / `project_channel`
Slack listener never starts, no warning printed	`"slack"` missing from `notify_channels`	`_start_slack_listener` returns silently in this case by design — check config first
Slack listener warns about missing env vars	`DEVTEAM_SLACK_APP_TOKEN` / `DEVTEAM_SLACK_TOKEN` not set in the process that runs the supervisor	Confirm the env vars are in the *supervisor's* shell/PM2 config, not just your interactive shell
Tower warnings every tick	Expected if Tower is down/unreachable — this is the fail-open design working correctly, not a bug	Confirm exactly ONE warning per tick, not a flood, not a crash
A command from Slack/Tower silently does nothing	Check `.devteam/inbox/rejected/*.reason` — it was likely rejected (unknown command, bad schema, duplicate id)	Read the `.reason` sidecar
Duplicate command re-executes	Ledger (`.consumed_ids.json` under `.devteam/inbox/`) missing/deleted, or a non-standard inbox path used	Confirm the ledger file exists and is being written to the same `.devteam/inbox/` the drain reads from

---

Sign-off checklist

- ☐ Suite A green (pytest + node)
- ☐ Suite B: known-good accepted, unknown + typo both rejected (never guessed)
- ☐ Suite C.1 no-op confirmed; Suite C.2's five-step live checklist completed on a test project
- ☐ Suite D.1 absent-dependency fail-open confirmed; D.2 live round-trip confirmed (if `slack_sdk` in use)
- ☐ Suite E.1 push/pull round-trip confirmed against the stub; E.2 fail-open (exactly one warning, no crash) confirmed
- ☐ Suite F.1–F.4 all pass (happy path, rejection, duplicate, crash-recovery)
- ☐ Suite G.1 disabled-parity confirmed; G.2 full integration confirmed
- ☐ Rollback steps tested at least once (flip both `enabled` flags off, confirm clean no-op)

Only once every box above is checked should `tower.enabled` / `slack.enabled` be flipped to `true` on a real project.